Exercises: Introductory Course on Implicitly Switched ODEs and Filippov Sliding

Andreas Sommer, Lev Gromov, Marvin Hertweck, Pilar Keller

July 09, 2026

Appendix: IFDIFF Cheat Sheet

RHS Function Requirements

- RHS functions must be implemented in a separate MATALB file, e.g. `myRhs.m`.
  The RHS function must have the signature `function dy = myRhs(t, y, p)`, i.e. it takes a timepoint, state vector and parameter vector and returns the derivatives of the state vector as a column vector.
- Helper functions, which the RHS function calls internally, may be defined in separate function files or as nested functions, but **do NOT** work as local functions of `myRhs.m`. Nested functions **must NOT** contain switches.

IFDIFF Solution Setup: A Canonical Example

We consider the following switched ODE

$$\frac{d}{dt}y(t) = f(t, y, p) = \begin{bmatrix} f_1(t, y, p) \\ f_2(t, y, p) \end{bmatrix}$$

where

$$f_1(t, y, p) = 0.01t^2 + y_2^3$$

$$f_2(t, y, p) = \begin{cases} 0, & \text{if } y_1 < p \\ 5, & \text{if } p \leq y_1 < p + 0.5 \\ 0, & \text{if } y_1 \geq p + 0.5 \end{cases}$$

with initial conditions $y_1(0) = 0$, $y_2(0) = 1$ and parameter $p = 5.437$ over the time interval $I = [0, 20]$.

1. Implement the RHS as a function in a new MATLAB file `rhsCanonicalExample.m`

```
function dy = rhsCanonicalExample(t, y, p)
dy = zeros(2,1);
dy(1) = 0.01*t.^2 + y(2).^3;

if y(1) < p(1)
    dy(2) = 0;
elseif y(1) < p(1) + 0.5
    dy(2) = 5;
else
    dy(2) = 0;
end
end
```

2. Setup the initial values, time span and parameters in your main script `runCanonicalExample.m`.

```
tspan = [0, 20];
y0 = [1; 0];
p = 5.437;
```

3. Set the RHS, the integrator and its options in the main script. Call `prepareDatahandleForIntegration` to prepare the RHS for integration with IFDIFF.

```
rhs = @rhsCanonicalExample;
integrator = @ode45;
options = odeset('RelTol', 1e-10, 'AbsTol', 1e-10);

datahandle = prepareDatahandleForIntegration(rhs, ...
    'integrator', integrator, 'options', options);
```

4. Solve the initial value problem by calling `solveODE` to obtain a solution struct.

```
solution = solveODE(datahandle, tspan, y0, p);
```

5. Create a vector of timepoints and evaluate the solution at these timepoints using `deval`. You can then plot the solution.

```
t = linspace(tspan(1), tspan(end), 1000);
y = deval(solution, t);
plot(t, y);
```

6. To compute sensitivities, first create a sensitivity function by calling `generateSensitivityFunction`. Afterwards, you can evaluate the created sensitivity function by passing a vector of timepoints to it.

```
sensitivityFunction = generateSensitivityFunction(datahandle, solution);
sensitivity = sensitivityFunction(t);
```

   The sensitivity function will return an array of structs of the same length as the number of timepoints. Each struct contains the fields `t`, `Gy` and `Gp` which store a timepoint and the initial value and parameter sensitivity matrix at that timepoint.

7. To plot the sensitivities, we strongly recommend that you use the provided helper functions in the respective exercises.

   If you want to plot the sensitivites on your own, you should first extract the timepoints and sensitivity matrices as arrays.

```
dimY = numel(y0);
dimP = numel(p);
t = [sensitivity.t];
Gy = reshape([sensitivity.Gy], dimY, dimY, []);
Gp = reshape([sensitivity.Gp], dimY, dimP, []);
```

   You can then plot the individual components of the matrices in a tiled plot

```
figure;
tiledlayout;
for idxRow=1:dimY
    for idxCol=1:dimY
        nexttile;
        plot(t, squeeze(Gy(idxRow, idxCol, :)));
    end
end

figure;
tiledlayout;
for idxRow=1:dimY
    for idxCol=1:dimP
        nexttile;
        plot(t, squeeze(Gp(idxRow, idxCol, :)));
    end
end
```

Specifying state jumps in a RHS

A state jump may be specified at any point in the RHS via a special if-block:

```
function dy = myRHS(t, y, p)
...
if ifdiff_jumpif(jumpCondition, jumpDirection)
... <statements computing the jumpIncrement> ...
ifdiff_update(jumpIncrement)
end
...
end
```

When the first argument of the `ifdiff_jumpif` function (`jumpCondition`) crosses zero in the direction specified by the second argument (`jumpDirection`), i.e. from negative to positive (1), from positive to negative (-1) or in any direction (0), IFDIFF will update the solution state vector by adding the jump increment defined in the first argument of the `ifdiff_update` function (`jumpIncrement`) to the solution state vector.

Important notes:

- The state vector and `jumpIncrement` must have matching dimensions. In particular, the `jumpIncrement` must be a column-vector with the same number of elements as the state vector.
- IFDIFF will never execute the body of the if-block directly in the RHS, so make sure that the statements in the if-block are only used to compute the `jumpIncrement`. In particular, any variables exclusively defined in the if-block must not be used outside of the if-block.

For reference, here is an implementation of the RHS for a bouncing ball. The first solution component represents the vertical distance of the ball from the ground and the second solution component the vertical velocity of the ball. The state jump triggers, when the ball hits the ground (i.e. when the first component becomes zero), causing the velocity to be inverted (with some loss of energy).

```
function dy = rhsBounceball(t, y, p)
dy = [y(2); -9.81];

if ifdiff_jumpif(y(1), -1)
    jump = [0; -(1 + 0.9)*y(2)];
    ifdiff_update(jump);
end
end
```